Ex-6

Python Implementation of Hamming Code Concept.

Aim:

Hamming Code is a set of error-corrections codes that can be used to detect & correct the errors when the data is transmitted from the sender to the receiver. To implement error detection and correction using Hamming code technique at Data Link Layer.

Sender Program:

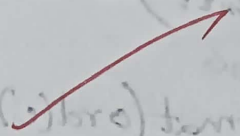
→ Takes text input

→ Converts text to binary

→ Divides binary data into 4-bit groups.

→ Adds 3 redundant parity bits using Hamming Code

→ Saves the encoded data in channel.txt



Scanned with OKEN Scanner

Code:

```
def hamming_encode(data):
    encoded = ""
for i in range(0, len(data), 4):
    d = data[i:i+4]
    if len(d) < 4:
        d = d.adjust(4, '0')
    d1, d2, d3, d4 = map(int, d)
    p1 = d1 ^ d2 ^ d4
    p2 = d1 ^ d3 ^ d4
    p4 = d2 ^ d3 ^ d4
    code = str(p1) + str(p2) + str(d1) +
str(p4) + str(d2) + str(d3) + str(d4)
    encoded += code
return encoded.

text = input("Enter Text:")
binary = ''.join(format(ord(c), '08b') for
                                c in text)
print("Binary data:", binary)
encoded_data = hamming_encode(binary)
with open("channel.txt", "w") as file:
    file.write(encoded_data)
print("Hamming encoded data :", encoded_data).
```

Scanned with OKEN Scanner

Receiver Code:-

→ Reads data from channel.txt.

→ Checks every 7-bit hamming block.

→ Finds the error position using syndrome.

→ Corrects the errors & removes parity bits.

→ Converts binary back to ASCII text.

Code:-

```
def hamming_decode(data):
    corrected = ""
    error_found = False

    for i in range(0, len(data), 7):
        block = list(map(int, data[i:i+7]))
        if len(block) < 7:
            continue
        p1, p2, d1, p4, d2, d3, d4 = block
        s1 = p1 ^ d1 ^ d2 ^ d4
        s2 = p2 ^ d1 ^ d3 ^ d4
        s4 = p4 ^ d2 ^ d3 ^ d4
        error_position = s1 + (s2*2) + (s4*4)

        if error_position != 0:
            error_found = True
```

```
        actual_position = i + error_position
        print("Error detected at bit
                                position:", actual_p
        block[error_position - 1] ^= 1
        print("Error corrected at bit position:",
                                    actual_posit
    corrected += str(block[2])
    corrected += str(block[4])
    corrected += str(block[5])
    corrected += str(block[6])

    return corrected, error_found

with open("channel.txt", "r") as file:
    channel_data = file.read().strip()
print("Data received from channel:",
                                channel_data)
binary_data, error_found = hamming_decode(channel_
                                            data
text = ""
```

Scanned with OKEN Scanner

```
for i in range (0, len(binary_data), 8):
    byte = binary_data [i : i+8]
    if len (byte) == 8 :
        text += chr (int (byte, 2))
print ("\n Corrected binary data:")
print (binary_data)
if (error_found):
    print ("\n Error was detected and corrected.")
else:
    print ("\n No Error detected.")
print ("Received text :", text)
```

Input:-

Enter the text : HELLO

Out put:-

Binary data : 0100100001000101001000011000
1001111

Hamming encoded data : 10011001110000010011000 10
10111000111100100111111

Data saved to channel.txt

Result:-

Thus The hamming code was succesfully implemented to detect and correct errors in the transmitted data.

Scanned with OKEN Scanner